3 Attribute Shadowing



KEY IDEAS

- Assigning to an instance attribute whose name also exists on the class creates a **new instance attribute** that *shadows* the class one. The class attribute is untouched.
- `del obj.attr` removes the shadow; the class value shows through again.
- Trap: `self.n += 1` **reads** the class attribute but **writes** an instance attribute.

> **TIP** To bump shared state from an instance method use `type(self).n += 1` (or `ClassName.n += 1`).

EXAMPLE

```
class Car:
    wheels = 4

a, b = Car(), Car()
a.wheels = 6                    # shadows the class attr
print(a.wheels, b.wheels)       # → 6 4
print(Car.wheels)               # → 4  (unchanged)
print(a.__dict__)               # → {'wheels': 6}
del a.wheels                    # remove the shadow
print(a.wheels)                 # → 4
```

```
class Cnt:
    n = 0
    def __init__(self):
        self.n += 1         # BUG: shadows; Cnt.n stays 0
        # Cnt.n += 1        # FIX: update the class attr

x, y = Cnt(), Cnt()
print(Cnt.n, x.n, y.n)      # → 0 1 1
```